

PLAN DE MVP

Motor automatico de videos cortos educativos

Formato TikTok / Instagram Reels con Teto, Miku, Luka, Rin y Len

Objetivo del documento

Especificacion accionable para entregarla a un agente de Claude Code y llevar un proyecto desde cero hasta un MVP tecnico capaz de producir un video vertical de 60-120 segundos a partir de guion, audio, avatares JPG, recursos visuales y video de fondo.

Fecha: 30 de septiembre de 2026

Version 1.0 - MVP tecnico

1. Decision clave: si, recopilar reacciones a mano

Para el MVP conviene recopilar manualmente un banco pequeno de JPG/PNG por personaje. No se necesita una biblioteca enorme: el objetivo es demostrar que el motor puede elegir una reaccion, mostrarla en el momento correcto y mantener una identidad visual consistente.

Recomendacion practica

No intentes conseguir 30-50 expresiones por personaje. Empieza con 8 estados por personaje y agrega nuevas reacciones solo cuando un guion real las necesite.

Banco inicial por personaje

Estado	Uso narrativo	Prioridad	Ejemplo de nombre
neutral	explicacion normal / escucha	Alta	Luka_neutral.png
feliz	aprobacion / remate ligero	Alta	Luka_feliz.png
sorprendido	hook / dato inesperado	Alta	Luka_sorprendida.png
confundido	pregunta / malentendido	Alta	Luka_confundido.png
enojado	correccion / frustracion humoristica	Media	Luka_enojado.png
riendo	remate / reaccion	Media	Luka_riendo.png
serio_nerd	explicacion tecnica	Media	Luka_nerd.png
shocked	giro exagerado / meme	Media	Luka_shocked.png

Por que 8 estados funcionan para el MVP

- Cubren la mayoria de los actos narrativos del formato: explicar, preguntar, reaccionar, corregir y rematar.
- Permiten que el motor haga cambios visibles sin necesitar animacion de personaje.
- La IA puede aprender una taxonomia simple y predecible antes de manejar assets mas complejos.
- La biblioteca se puede ampliar incrementalmente sin cambiar la arquitectura.

Importante: usa nombres estandarizados y evita variantes ambiguas. Por ejemplo, preferir Luka_sorprendida.png sobre mezclas como Luka_sorpresa2.jpg, Luka_surprised.jpg y Luka_sorpresa_final.png.

2. Alcance del MVP tecnico

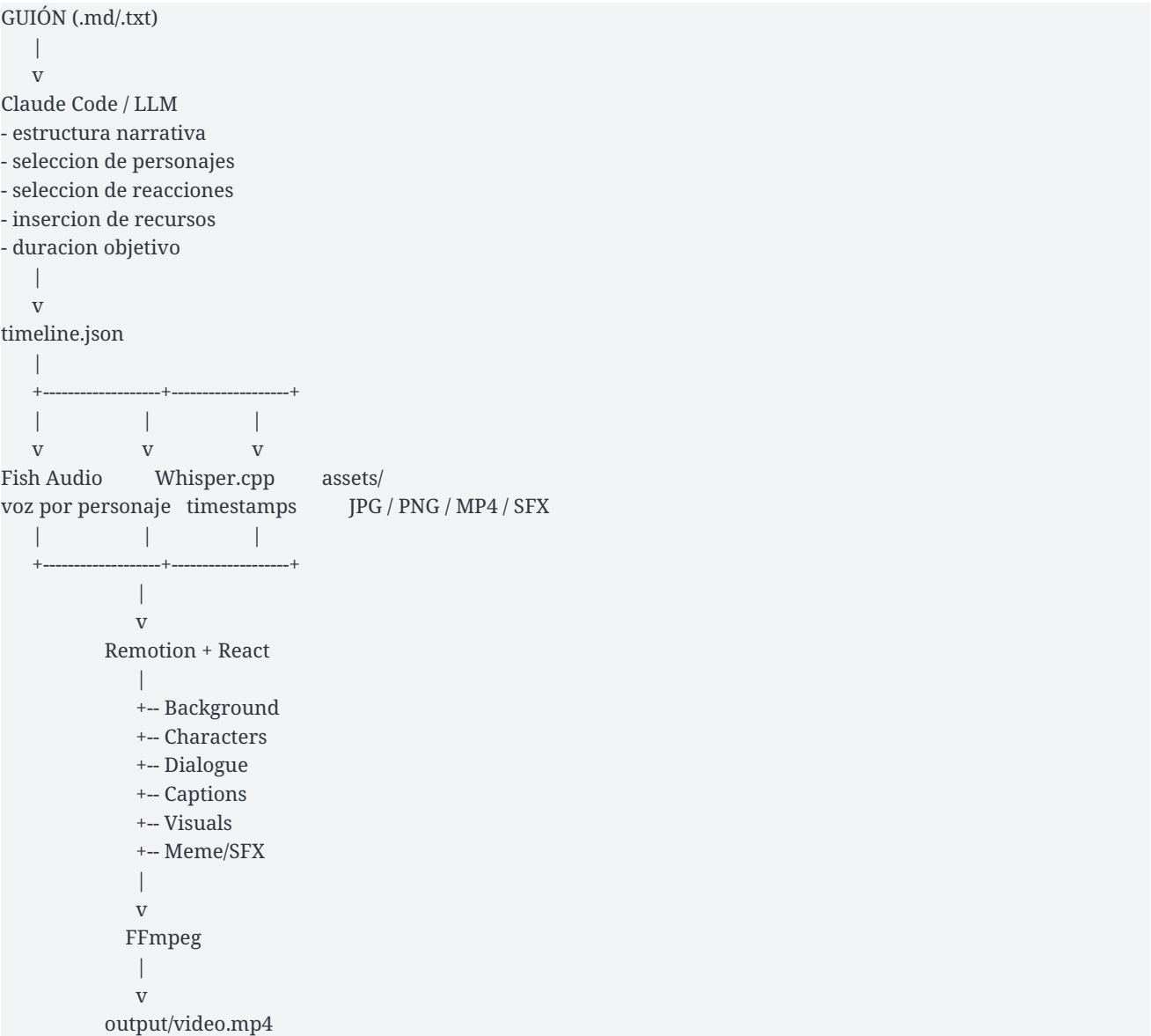
El MVP debe demostrar un flujo extremo a extremo con un video de 60 a 120 segundos. El objetivo no es crear un editor completo ni una plataforma web, sino un generador reproducible y configurable por archivos.

Incluido	Fuera del MVP	Criterio de inclusion
Video 1080x1920, 30 fps	Generacion de clips AI complejos	No es necesario para probar el formato
Audio TTS desde Fish Audio o WAV/MP3 ya generado	Lip-sync facial	El avatar es estatico
Avatares JPG/PNG por emocion	Personajes 3D	No aporta al MVP
Subtitulos por palabra o grupos de palabras	Editor drag-and-drop completo	Puede venir despues
Video MP4 de fondo	Backend cloud	Todo puede ejecutarse localmente
Imagenes/logos/diagramas insertables	Generacion automatica de imagenes	Usar assets ya disponibles
SFX simples y eventos meme	Sistema de recomendacion de memes	Manual primero
Validacion estricta de 60-120 s	Publicacion directa a TikTok/Instagram	La exportacion es suficiente
Preview en Remotion Studio	Autopublicacion	Segunda etapa

Definicion de exito

Dado un proyecto con guion + audio + catalogo de assets, el sistema produce automaticamente un MP4 vertical de al menos 60 segundos, con dialogo, cambios de avatar, subtitulos con color por personaje, recursos visuales y una secuencia narrativa coherente.

3. Arquitectura propuesta



Responsabilidades

Capa	Responsabilidad	Regla
Claude Code	Construir el proyecto, ejecutar scripts, probar, depurar y mantener el motor.	No renderiza visualmente por su cuenta; implementa el pipeline.
LLM	Transformar el guion a eventos estructurados.	Salida estrictamente JSON validable.
Fish Audio	Generar voces por personaje.	Un archivo de audio por intervencion o por bloque de dialogo.

Capa	Responsabilidad	Regla
Whisper.cpp	Obtener timestamps de palabras.	Local, reproducible, sin depender de API para el MVP.
Remotion	Componer el video.	La logica visual vive en React/TS.
FFmpeg	Codificacion, mux, normalizacion y operaciones finales.	No usarlo como sustituto de la logica de escenas.

Principio de arquitectura

La IA decide QUE debe ocurrir; el motor determina COMO ocurre. El LLM no debe tener libertad pixel a pixel. Esto hace el sistema estable, testeable y reutilizable.

4. Sistema de assets y catalogo de reacciones

Cada asset debe estar acompañado por metadatos. El agente no debería escanear imagenes y adivinar su significado cada vez; debe consultar un catalogo estructurado.

```
assets/
characters/
  teto/
    neutral.png
    feliz.png
    sorprendida.png
    confundida.png
    enojada.png
    riendo.png
    nerd.png
    shocked.png
  miku/
  luka/
  rin/
  len/
backgrounds/
visuals/
logos/
memes/
sfx/

config/characters.json
config/assets.json
```

Formato recomendado para characters.json

```
{
  "teto": {
    "displayName": "Teto",
    "subtitleColor": "#E58FB5",
    "defaultScale": 0.42,
    "anchor": "bottom-left"
  },
  "miku": {
    "displayName": "Miku",
    "subtitleColor": "#39C5BB",
    "defaultScale": 0.42,
    "anchor": "bottom-left"
  }
}
```

Los colores anteriores son una propuesta de implementacion para el MVP, no una declaracion de colores oficiales. Ajustarlos a partir del material visual concreto que utilices.

Taxonomia visual minima

- neutral: habla o escucha sin énfasis.
- feliz / riendo: acuerdo, alivio, chiste.
- sorprendida / shocked: dato fuerte, hook, giro, meme.
- confundida: pregunta, contradicción, incomprensión deliberada.
- enojada: corrección, frustración cómica.

- nerd / seria: explicacion tecnica, conclusion, dato riguroso.

Catalogo de assets visuales

```
{
  "id": "deepseek_logo",
  "type": "image",
  "path": "visuals/deepseek_logo.png",
  "tags": ["ia", "empresa", "logo"],
  "safeArea": true,
  "license": "documentada",
  "source": "..."
}
```

Regla: toda imagen externa debe registrar al menos source y license_status. Si el estado es unknown, el agente puede usarla en un render de prueba local, pero debe marcarla como bloqueada para una publicacion comercial.

5. Lenguaje audiovisual del canal

El formato debe ser reconocible incluso si cambia el tema. La consistencia debe provenir de reglas del motor, no de instrucciones manuales en cada video.

Reglas de personajes

- El personaje que habla recibe prioridad de escala y posicion.
- El personaje que escucha puede aparecer en una escala menor o con una expresion de reaccion.
- No mostrar a los cinco personajes simultaneamente salvo que el evento lo justifique.
- Un cambio de avatar debe coincidir con un cambio semantico, una reaccion o un punchline, no cada palabra.

Reglas de subtítulos

Personaje	Color sugerido	Uso
Teto	#E58FB5	rosa
Miku	#39C5BB	turquesa
Luka	#E285B2	rosa/magenta
Rin	#F6C744	amarillo
Len	#F2B544	amarillo/naranja

La identidad visual de cada personaje debe almacenarse en characters.json para que ningun guion pueda cambiar accidentalmente su color.

Estructura narrativa base

1. Hook de 2-5 segundos con una afirmacion/pregunta fuerte.
2. Reaccion visual o meme de 1-4 segundos.
3. Contexto tecnico inicial.
4. Interrupciones o storyline entre personajes.
5. Insercion de imagenes, graficos o logos cuando se introduce cada concepto.
6. Remate o giro humoristico.
7. Cierre que resume la idea y deja una ultima reaccion.

Restriccion de duracion

```

MIN_DURATION_MS = 60_000
MAX_DURATION_MS = 120_000
TARGET_DURATION_MS = 85_000

if duration < MIN_DURATION_MS:
    extend_scene_or_add_explanation()
if duration > MAX_DURATION_MS:
    request_recompression_of_timeline()

```

6. Contrato de datos: timeline.json

Este archivo es el contrato entre la inteligencia narrativa y el renderizador. Debe ser JSON puro, validado con un esquema JSON y sin funciones.

```
{
  "meta": {
    "title": "China no destruyo a ChatGPT",
    "durationTargetSec": 85,
    "aspect": "9:16",
    "fps": 30
  },
  "scenes": [
    {
      "id": "hook",
      "startMs": 0,
      "endMs": 4200,
      "character": "teto",
      "avatar": "sorprendida",
      "dialogue": "¿China destruyo a ChatGPT y Claude?",
      "visuals": ["chatgpt_logo", "claude_logo"],
      "events": ["subtitle_emphasis"]
    },
    {
      "id": "meme",
      "startMs": 4200,
      "endMs": 6500,
      "events": ["meme_explosion"]
    }
  ]
}
```

Tipos de eventos del MVP

character_reaction - Cambia el avatar de un personaje.

visual_show - Muestra una imagen/logo/grafico durante un intervalo.

visual_hide - Retira un recurso antes del fin de la escena.

camera_zoom - Zoom suave para enfatizar una idea.

camera_shake - Sacudida breve para meme o shock.

meme_explosion - Preset combinado de flash, SFX, escala y corte.

subtitle_emphasis - Resalta una o varias palabras.

pause - Deja una micro-pausa comica si el audio lo permite.

Regla de consistencia

El timeline debe poder renderizarse dos veces y producir el mismo resultado visual, salvo que el usuario cambie explicitamente los assets o parametros.

7. Pipeline de generacion

Paso	Accion	Resultado
1	1. Ingreso	Recibir guion, assets disponibles y opcionalmente audio ya creado.
2	2. Analisis narrativo	Claude extrae personajes, dialogos, eventos, recursos y ritmo.
3	3. Validacion	Comprobar personajes existentes, avatars validos, assets encontrados y duracion objetivo.
4	4. Generacion de voz	Fish Audio genera cada intervencion segun el personaje y la configuracion de voz.
5	5. Concatenacion de audio	Construir una pista maestra y conservar un indice de bloques por personaje.
6	6. Transcripcion	Whisper.cpp calcula timestamps para subtítulos.
7	7. Reajuste	Asignar duraciones reales a cada escena usando audio/timestamps.
8	8. Render	Remotion compone el video con background, avatares, visuales, captions y SFX.
9	9. Validacion final	Verificar 60-120 s, 1080x1920, audio sincronizado, subtítulos y ausencia de assets faltantes.
10	10. Salida	Guardar MP4, timeline.json, subtitles.srt y un reporte de validacion.

Una recomendacion tecnica importante

Para el MVP no intentes calcular el timing exacto de cada escena unicamente con la estimacion del LLM. El audio generado y su transcripcion deben ser la autoridad temporal. La IA puede proponer la estructura; el pipeline corrige las duraciones despues de obtener el audio real.

8. Estructura del repositorio

```

short-video-engine/
├── assets/
│   ├── characters/{teto,miku,luka,rin,len}/
│   ├── backgrounds/
│   ├── visuals/
│   ├── logos/
│   ├── memes/
│   └── sfx/
├── config/
│   ├── characters.json
│   ├── assets.json
│   └── render.json
├── projects/
│   └── demo_001/
│       ├── script.md
│       ├── timeline.json
│       ├── audio/
│       ├── transcript/
│       └── report.json
├── src/
│   ├── compositions/
│   ├── components/
│   ├── timeline/
│   ├── validation/
│   └── utils/
├── scripts/
│   ├── analyze-script.ts
│   ├── generate-voices.ts
│   ├── transcribe.ts
│   ├── build-timeline.ts
│   ├── validate.ts
│   └── render.ts
├── schemas/
│   ├── timeline.schema.json
│   ├── characters.schema.json
│   └── assets.schema.json
├── tests/
└── output/

```

Tecnologías

Tecnología	Version inicial	Motivo
TypeScript + Node.js	LTS actual	Scripts, tipado y automatización.
React + Remotion	actual	Composición frame-by-frame y preview.
Whisper.cpp	actual	Transcripción local y timestamps.
FFmpeg	actual	Muxing, codecs, audio y postproceso.
Fish Audio API	actual	TTS por personaje cuando se

disponga de una voz autorizada.

Claude Code	actual	Agente de desarrollo, tests, scripts y mantenimiento.
JSON Schema	estable	Validacion del contrato timeline.
Git	actual	Versionado del motor y de los presets, no necesariamente de assets pesados.

9. Plan de implementacion por fases

Fase	Implementar	Definition of done
Fase 0 - Preparacion	Crear repo, instalar Node/Remotion/FFmpeg, verificar entorno, crear CLAUDE.md y schemas.	Repo ejecuta un hello-world y renderiza un clip de 3-5 s.
Fase 1 - Catalogo	Normalizar nombres, crear characters.json y assets.json, validar assets faltantes.	El sistema puede resolver cualquier personaje/reaccion por ID.
Fase 2 - Render minimo	Construir Background, Character, Caption y Visual components.	Un timeline manual produce un MP4 9:16 correcto.
Fase 3 - Audio	Integrar Fish Audio con una interfaz abstracta TTSPProvider.	Un proyecto puede generar voces por bloque.
Fase 4 - Subtitulos	Integrar Whisper.cpp, convertir timestamps a captions y aplicar color por personaje.	Subtitulos sincronizados y legibles.
Fase 5 - Motor de timeline	Implementar eventos, presets, entrada/salida de assets y validadores.	timeline.json completo se renderiza sin edicion manual.
Fase 6 - Director LLM	Crear analyze-script y prompts/esquema para generar timeline.json.	Un guion nuevo genera un timeline valido.
Fase 7 - Validacion y demo	Agregar test automaticos, demo de 60-90 s y reporte.	MVP pasa todos los criterios de aceptacion.

Orden recomendado

No empezar por la integracion del LLM. Primero demostrar que un timeline escrito a mano puede renderizarse bien. Luego automatizar la creacion del timeline. Esto reduce dramaticamente el espacio de debugging.

10. Instrucciones para Claude Code

Este bloque puede convertirse en el contenido base de CLAUDE.md y en las primeras instrucciones del agente. Claude Code esta pensado para trabajar sobre el repositorio, ejecutar comandos, escribir y probar código, y mantener contexto del proyecto mediante CLAUDE.md. Referencias oficiales al final del documento.

```
# CLAUDE.md - resumen operativo
```

Objetivo:

Construir un motor local que transforme un guion en un video vertical educativo de 60-120s usando personajes Vocaloid representados por imagenes estaticas, subtítulos por color, recursos visuales y un fondo.

Reglas:

1. No inventar paths de assets.
2. No aceptar timeline invalido.
3. No permitir duracion < 60s ni > 120s.
4. No acoplar la logica del motor a Fish Audio.
5. No acoplar el motor a un proveedor concreto de LLM.
6. Todo componente importante debe tener prueba o fixture.
7. Preferir determinismo a comportamiento magico.
8. Antes de cambiar arquitectura, revisar schemas y tests.
9. Ejecutar lint/test/render-smoke despues de cambios relevantes.
10. Mantener una separacion clara entre narrativa y rendering.

Modo de trabajo recomendado para el agente

8. Inspeccionar el repo y escribir un plan antes de crear archivos importantes.
9. Implementar en incrementos pequenos y ejecutables.
10. Tras cada fase, ejecutar pruebas y un smoke render.
11. Cuando haya un defecto visual, modificar el componente responsable en lugar de introducir hacks en el timeline.
12. Mantener un CHANGELOG.md o ADR corto para decisiones estructurales.
13. Nunca eliminar tests para conseguir un render verde.

Comando objetivo final

```
npm run generate -- --project projects/demo_001
# o
npm run render -- --project projects/demo_001
```

El agente debe documentar exactamente que hace cada comando y mantener un comando separado para validar sin renderizar.

11. Ejemplo de proyecto piloto

El primer piloto debe ser deliberadamente pequeño: un tema de informatica, 2-3 personajes, 5-8 assets visuales, un video de fondo, 1-3 efectos meme y una duracion objetivo de aproximadamente 75-90 segundos.

Input

```
projects/demo_001/
script.md
background.mp4
visuals/
requested_voices.json
```

Guion de referencia

```
[TETO:surprised]
¿China destruyo a ChatGPT y Claude?

[MIKU:confused]
¿Pero de donde sacaste eso?

[SFX:meme_explosion]

[TETO:nerd]
Bueno... el titular exagera bastante.
El punto real es otro: la competencia en modelos de IA
esta cambiando muy rapido.

[MIKU:shocked]
Eso suena mucho menos dramatico.

[TETO:happy]
Si, pero es bastante mas interesante.
```

Timeline esperado

```
0-4.5s  Teto sorprendida + logos + hook
4.5-6.5s meme_explosion
6.5-18s Teto explica
18-23s  Miku interrumpe con confusion
23-45s  grafico / logos / comparacion
45-52s  reaccion humoristica
52-72s  explicacion tecnica
72-80s  remate y cierre
```

Duracion final esperada: 75-90s

Criterio de calidad del piloto

El espectador debe poder entender el concepto incluso sin leer el guion, reconocer quien habla por color/subtitulo y reaccion visual, y percibir una secuencia narrativa intencional.

12. QA y criterios de aceptacion

Check	Condicion	Tipo
Duracion	60.0 <= duration <= 120.0 segundos	Hard fail
Formato	1080x1920, 9:16, H.264/AAC o equivalente compatible	Hard fail
Audio	Sin clipping severo, voz inteligible, sync razonable	Hard fail
Subtitulos	Sin texto fuera de safe area; color correcto por personaje	Hard fail
Avatar	Cada personaje usa una reaccion valida del catalogo	Hard fail
Assets	No existen paths rotos ni imagenes vacias	Hard fail
Narrativa	Existe hook + desarrollo + cierre	Hard fail
Humor	Al menos una interaccion/reaccion/punchline intencional en el piloto	Soft fail
Performance	El render local termina sin errores y genera MP4 reproducible	Hard fail
Reproducibilidad	Mismo timeline + mismos assets produce mismo resultado	Hard fail

Test automaticos minimos

- Schema validation para timeline, characters y assets.
- Pruebas unitarias para conversion de timestamps a frames.
- Pruebas para duracion minima/maxima.
- Fixture de render de 5-10 segundos para CI/local smoke test.
- Prueba de resolver asset por ID.
- Prueba de color de subtitulo por personaje.

Validacion visual manual del MVP

- Abrir el render completo y revisar hook, transiciones, meme y cierre.
- Revisar subtitulos en pantalla de telefono o preview equivalente.
- Comprobar que los cambios de avatar tienen intencion narrativa y no parecen aleatorios.

13. Presupuesto y licencias para el MVP

El MVP tecnico puede plantearse con costo adicional cercano a cero si el desarrollo se hace en local y ya se dispone de un plan de Claude. Los costos aparecen principalmente al pasar de prototipo a produccion, por voz, assets y licencias concretas.

Componente	MVP local	Produccion inicial	Observacion
Claude Code	ya cubierto por el plan del usuario	idem	Usarlo como agente del repo, no como editor manual de clips.
Remotion	\$0 si se cumple Free License	depende del tamano del equipo/automatizacion	La licencia actual es gratuita para individuos y equipos de hasta 3 personas, con uso comercial y automatizacion permitidos bajo sus terminos.
Fish Audio	\$0 posible durante prototipo	pago por uso / plan	La web de desarrolladores indica S2.1 Pro Free para desarrolladores; verificar siempre derechos de uso comercial de la voz concreta y del plan.
Whisper.cpp	\$0	\$0	Local.
FFmpeg	\$0	\$0	Revisar configuracion/licencia si el software se distribuye.
Storage	\$0 local	bajo	Sube si se archivan muchos renders.
Assets/characters	variable	variable	El costo no es tecnico; depende de derechos de uso y de las ilustraciones concretas.

Licencias: no mezclar "puedo usar el personaje" con "puedo usar esta imagen/voz"

Para Miku, Rin, Len y Luka, la pagina oficial de Crypton indica CC BY-NC para las ilustraciones originales de los personajes; otras ilustraciones y obras pueden tener derechos distintos. Para Teto, las

guias oficiales permiten a particulares publicar videos y monetizarlos bajo determinadas condiciones, pero las reglas de voz y los usos comerciales deben revisarse por separado. El MVP puede ser tecnico/local, pero cualquier monetizacion debe pasar un chequeo de licencia de assets y voces concretos.

Fuentes actuales consultadas: Anthropic Claude Code; Remotion License & Pricing; Remotion captions; Fish Audio Developers/Pricing; Crypton Piapro character licensing; Kasane Teto official guidelines.

14. Entrega al agente: paquete inicial

Antes de iniciar el desarrollo, preparar esta estructura minima y pasarsela al agente en una sola sesion. El objetivo es reducir decisiones abiertas.

```
00_README.md
01_PRODUCT_SPEC.md    <- este plan, resumido si se desea
02_CHARACTER_RULES.md
03_ASSET_RULES.md
04_TIMELINE_SCHEMA.json
05_RENDER_RULES.md
assets/
  characters/miku/
  characters/luka/
  characters/teto/
  characters/rin/
  characters/len/
  visuals/
  memes/
  sfx/
projects/demo_001/
  script.md
  background.mp4
```

Checklist de arranque para Claude Code

14. Inspecciona el entorno y confirma Node, FFmpeg y Git.
15. Inicializa el proyecto TypeScript + Remotion.
16. Crea schemas y fixtures antes de implementar el generador narrativo.
17. Construye primero un render manual de 10 segundos.
18. Integra el sistema de personajes y subtítulos.
19. Integra transcripcion local.
20. Integra TTS mediante interfaz TTSPProvider.
21. Implementa el timeline y sus eventos.
22. Implementa el analizador de guion con salida JSON validada.
23. Construye y prueba demo_001 de 60-90 segundos.
24. Genera un reporte de que funciona, que falta y como ejecutar el pipeline.

Definition of Done del MVP

Existe un repositorio ejecutable desde una maquina local; un usuario coloca un guion y assets en un proyecto, ejecuta un comando, y recibe un MP4 vertical de 60-120 segundos con voces, subtítulos por color, cambios de avatar, visuales, fondo y al menos un evento de humor. El proceso deja tambien timeline.json, subtítulos y un reporte de validacion.

Referencias web

Anthropic - Claude Code CLI reference: <https://docs.anthropic.com/en/docs/claude-code/cli-usage>

Anthropic - Claude Code setup: <https://docs.anthropic.com/en/docs/claude-code/getting-started>

Remotion - Fundamentals: <https://www.remotion.dev/docs/the-fundamentals>

Remotion - TikTok-style captions: <https://www.remotion.dev/docs/captions/create-tiktok-style-captions>

Remotion - License & Pricing: <https://www.remotion.dev/docs/license/pricing>

Fish Audio - Developers: <https://fish.audio/developers/>

Fish Audio - Pricing: <https://fish.audio/plan/>

Crypton/Piapro - For Creators: https://piapro.net/intl/en_for_creators.html

Kasane Teto - Guidelines: <https://kasaneteto.jp/guidelines/>

Kasane Teto - Character guideline: <https://kasaneteto.jp/guidelines/character.html>

Kasane Teto - Voice guideline: <https://kasaneteto.jp/guidelines/voice.html>

Fin del plan - version 1.0